

DRAVA: An Event-Driven Runtime for Scientific Pipelines with Agentic Configuration Optimization

Abstract—

Next-generation X-ray science (XRS) detectors are driving continuous, multi-stage streaming pipelines with facility-scale data rates approaching petabytes per second. These workflows require real-time reconstruction, AI inference, and feedback-driven control under strict performance and energy constraints. The current software stack exposes scientists with manual tuning of batching, transport, and resource decisions.

We propose DRAVA, an event-driven runtime for scientific streaming pipelines. Developers express application stages as reusable components, while DRAVA manages event scheduling, batching, data movement, and transport-aware execution across heterogeneous resources. This separates scientific logic from system concerns and exposes runtime control over granularity, locality, and resource utilization.

We integrate DRAVA in multi-stage ptychography and tomography applications. It sustains high-rate streaming execution, reaching about 31 kHz on the runtime message path. Compared with PvaPy, a state-of-the-art streaming framework for real-time data processing, DRAVA handles unpaced input without dropping frames and achieves up to $2.36\times$ higher throughput. Its observability enables practical tuning: batching improves throughput by up to $2.44\times$, and agentic Bayesian optimization finds a configuration $2.24\times$ faster than the best manual one. DRAVA reports throughput–energy trade-offs, demonstrating observable and tunable software infrastructure for scientific pipelines on general-purpose high-performance computing (HPC) systems.

Index Terms—Scientific Pipelines, Runtime Systems, Event-Driven, HPC, XRS, Inference, Agentic workflow

I. INTRODUCTION

Scientific facilities in X-ray science (XRS) increasingly rely on always-on instruments that generate data faster than it can be transported, reduced, and analyzed. Facility-scale aggregation already places bandwidth pressure on data movement systems, while kHz-class acquisition requires near-real-time, multi-stage processing [1]. For example, the Advanced Photon Source (APS) Upgrade at Argonne National Laboratory increases X-ray beam brightness by up to $500\times$, enabling higher-resolution measurements and faster acquisition rates with data transfer requirements reaching tens of Gbit/s and reconstruction workloads approaching petascale compute [2]. As detector throughput continues to grow, future instruments are expected to drive data rates several orders of magnitude higher, intensifying challenges in data reduction and transport [3].

These scientific workflows must coordinate ingestion, pre-processing, inference, analysis, and feedback while experiments are running. Intermediate results can guide adaptive sampling or region-of-interest refinement [4], creating strict requirements for end-to-end latency and sustained throughput.

Custom detector pipelines and FPGA/ASIC-based systems can provide low latency [5], [6], [7], [8], [9], but are difficult to reconfigure as detectors, algorithms, and resource availability change [2].

Current software approaches remain fragmented. Frameworks such as PvaPy support EPICS pvAccess-based real-time streaming [10], but pipeline-wide control over scheduling, batching, placement, transport, and observability is often external or manually tuned [11], [12]. These decisions interact across stages, making static heuristics brittle under changing data rates, workloads, and resource conditions.

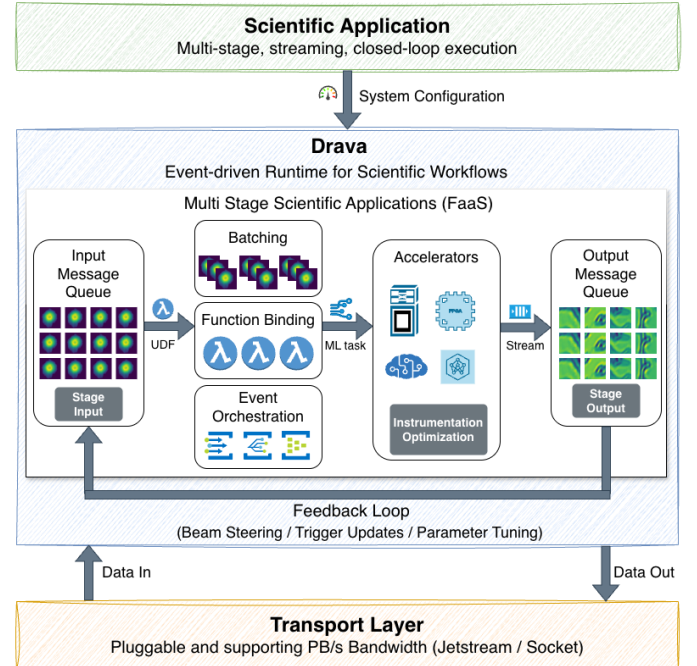


Fig. 1: Overview of DRAVA: multi-stage event-driven pipelines with runtime-managed batching, scheduling, transport, and observability across heterogeneous resources.

To address these challenges, we design and implement DRAVA, an event-driven runtime for scientific streaming pipelines offering a lightweight real-time serverless programming model implemented on top of XKRT [13], a state-of-the-art tasking runtime for heterogeneous architectures.

As summarized in Figure 1, developers express applications as reusable event-driven stages connected by data streams, while the runtime manages batching, scheduling, data movement, resource placement, and observability. DRAVA provides

transport-aware execution through a modular layer currently supporting NATS JetStream [14], which exposes a durable publish-subscribe model, and a low-latency socket transport, enabling flexible edge-to-HPC data movement.

DRAVA also targets the broader co-design need for system software that can expose performance–energy trade-offs across heterogeneous scientific computing resources [15], [16]. Related efforts such as SystemFlow model sensing and data-collection pipelines under power and relevance constraints [17], while UpDown explores architectural support for fine-grained, high-throughput computation and irregular data movement [18]. Recent EJFAT work shows a complementary transport-focused direction for moving detector streams from experimental facilities to remote HPC-hosted analysis workflows [19]. DRAVA complements these efforts with a runtime layer that measures throughput, latency, and energy, and uses guided configuration search to identify high-performing execution policies under limited sampling budgets.

Our main contributions are:

- An event-driven programming model—expressing multi-stage scientific workflows as streaming pipelines.
- An implementation in DRAVA: a runtime system that coordinates scheduling, batching, data movement, and heterogeneous resource placement across pipeline stages. DRAVA provides observability capabilities, with stage-level and pipeline-level measurements, exposing how runtime decisions affect throughput, latency, data movement, and energy use. It enables the analysis of workflow behavior as a complete system.
- An agentic configuration optimization—with a guided search framework that tunes runtime settings under a limited evaluation budget. Our evaluations explore only 0.52% of the configuration space and finds a configuration $2.24\times$ faster than the best manually selected one.
- Integration in scientific streaming workloads. We evaluate DRAVA on representative ptychography and tomography pipelines. Compared with a direct PvaPy baseline on the same PtychoNN inference stream, DRAVA remains loss-free at publisher rates where the baseline drops frames, while reaching $2.36\times$ higher throughput. The experiments show a runtime message-rate ceiling of about 31 kHz, up to $2.44\times$ throughput improvement from batching-aware tuning, and measured throughput–energy trade-offs across streaming inference configurations.

The remainder of this paper presents the runtime design (Section II), agentic optimization framework (Section III), performance evaluation (Section IV), energy analysis (Section V), related work (Section VI), and conclusion (Section VII).

II. SYSTEM DESIGN AND RUNTIME ARCHITECTURE

The design of DRAVA is guided to solve the challenges introduced in Section I. Rather than treating abstraction, configuration, execution, and observability as independent concerns, DRAVA co-designs these components within a unified runtime. The system is structured to (i) express streaming

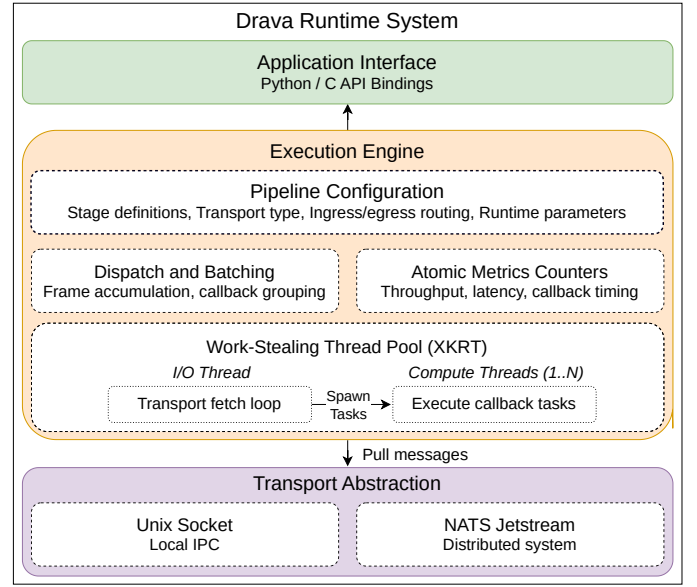


Fig. 2: Layered architecture of DRAVA. Application bindings, configuration, execution, and transport occupy distinct layers while sharing a unified event-processing path.

scientific workflows as event-driven compositions, (ii) expose system-level configuration without modifying application logic, (iii) sustain throughput under dynamic work distribution on available resources, and (iv) provide low-overhead, end-to-end observability for performance and energy analysis.

DRAVA is structured as a layered event-driven runtime that maps user-defined callbacks into configured, instrumented pipeline stages. Figure 2 shows the overall organization: application callbacks enter through C or Python bindings, a YAML configuration layer binds each stage to its transport, batching policy, and thread resources, and the execution engine manages I/O, microbatching, task dispatch, and metrics collection. The remainder of this section describes each layer from the programming interface down to the transport layer. This section introduces its programming model and each layer of its runtime system implementation.

A. Programming Model

DRAVA adopts an event-driven programming model. Each pipeline stage is a stateless function triggered by data events. This choice is motivated by the structure of streaming scientific workflows: stages differ in computational cost, input rate, and resource requirements, so execution must be driven by data availability rather than a global schedule. Stateless callbacks also decouple application logic from runtime policy, allowing the same scientific code to be evaluated under different batching, transport, and scheduling configurations.

A DRAVA workflow is a directed acyclic graph, where nodes represent *stages* and edges *transport routes*. Each stage is defined by four elements:

- 1) An *ingress route* for events entering the stage,
- 2) A *callback* that performs domain-specific computation,

- 3) An *egress route* for publishing derived events, and
- 4) *Runtime parameters* that control batching, scheduling, and thread resources.

The application specifies items 1)–3); item (4) is supplied by external configuration, so the same callback can be evaluated under different execution policies without modification.

B. The Transport Layer

DRAVA supports two transport backends: Unix domain sockets for low-latency local communication, and NATS JetStream [20] for distributed event streaming. Both interfaces receive messages (or a “frame”), each carrying a unique frame identifier, a receive timestamp, and a payload (i.e., the raw data).

The socket backend uses a Unix domain socket for ingress and a FIFO for egress. Each socket input frame is encoded as (i) a length encoded on a four-byte big-endian integer n , followed by (ii) the raw payload of n bytes. This backend provides a lightweight local transport for efficient in-place processing within a single compute node.

The NATS JetStream backend relies on a publisher/consumer message model [14]—a model that notoriously scales to distributed systems for processing a large set of messages. We rely on three concepts:

- *Messages*—the actual data sent (or “events” in DRAVA).
- *Subjects*—that is an abstract topic for typing messages.
- *Streams*—a storage for messages of specific subjects.

Messages are emitted by producers and read by consumers, who may themselves emit additional messages. Each stage specifies a subject and asynchronous streams for ingress and egress. For resilience purposes, each stage includes a *durable consumer*—a built-in of Jetstream that saves messages to persistent storage in case of crashes or disconnection.

DRAVA pull events from transport backends. On a predefined End-of-Stream (EoS) event, DRAVA drains outstanding asynchronous messages before it shuts down. Pulled messages are enqueued and processed by the *execution engine*.

C. Execution Engine

The execution engine is built on the XKRT [13] parallel runtime system. DRAVA forks a team of threads, where Thread 0 runs an I/O loop to pull messages from the transport layer, and Threads $1 \dots N-1$ form a compute-pool using efficient topology-aware and hierarchical work-stealing.

1) *Microbatching Messages*: Thread 0 accumulates messages to a pending buffer. The messages are flushed when any of three conditions is met:

- (i) *Threshold*: the pending count reaches the configured callback batch size B^{cb} .
- (ii) *End-of-stream*: an EoS message arrives, flushing and finalizing the stage.
- (iii) *Timeout*: a timer τ expires with $|P| > 0$ pending messages, bounding tail latency for low-rate streams.

Figure 3 shows the full dispatch flowchart that leads to flushing pending messages to callbacks. Threshold-based flushing

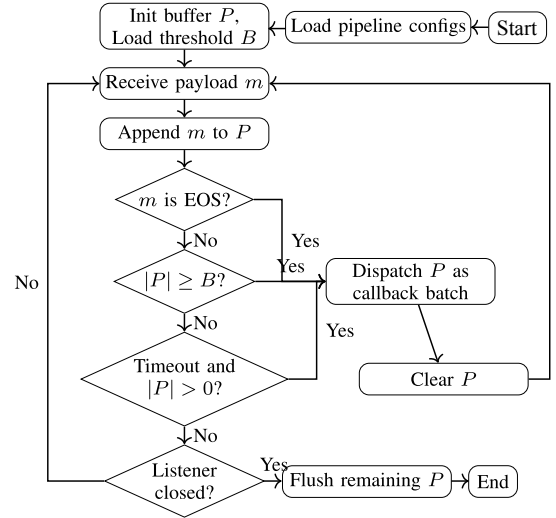


Fig. 3: Callback microbatching in DRAVA. Pending records are dispatched when EOS arrives, $|P|$ reaches threshold B , or a receive timeout occurs.

TABLE I: Per-stage metrics reported by DRAVA.

Metric	Meaning	Source
rx_msgs	Callback batches processed	Counter
rx_items	Non-EoS frames received	Counter
rx_bytes	Input payload bytes	Counter
tx_msgs	Published payloads	Publish
tx_bytes	Published bytes	Publish
cb_avg_ms	Mean callback-batch duration	Timer
publish_total_s	Cumulative publish time	Timer
stage_avg_ms	Mean dispatch-to-publish time	Samples
stage_total_fps	Frames per stage interval	Snapshot

amortizes per-callback overhead; timeout-based flushing prevents partial batches from stalling indefinitely. Because this logic is shared across both transport backends, batching is a first-class runtime policy: batch size, timeout, and transport interactions can be explored systematically without modifying the application callback.

2) *Flushing Batched Messages*: DRAVA provides two modes for flushing batch of messages: *parallel* and *serial*. When parallel flushing is enabled, each batch executes as an XKRT task, allowing idle workers to work-steal pending batches dynamically. When serial flushing is enabled, each worker takes a global mutex lock before executing the callbacks, ensuring at most only one thread executes at the same time—providing support to non thread-safe applications.

3) *Integrated Metrics Counters*: Additionally, the runtime provides an observability capability with built-in performance and energy metrics. All counters use lock-free for sums and a CAS loop for maxima; the first-receive timestamp is recorded with a one-shot CAS. These operations add negligible execution time overhead. Table I lists the metrics recorded per stage. For instance, the `cb_avg_ms` is the average time spent by each thread when executing a callback over a batch of events. The stage runtime interval spans the first receive to

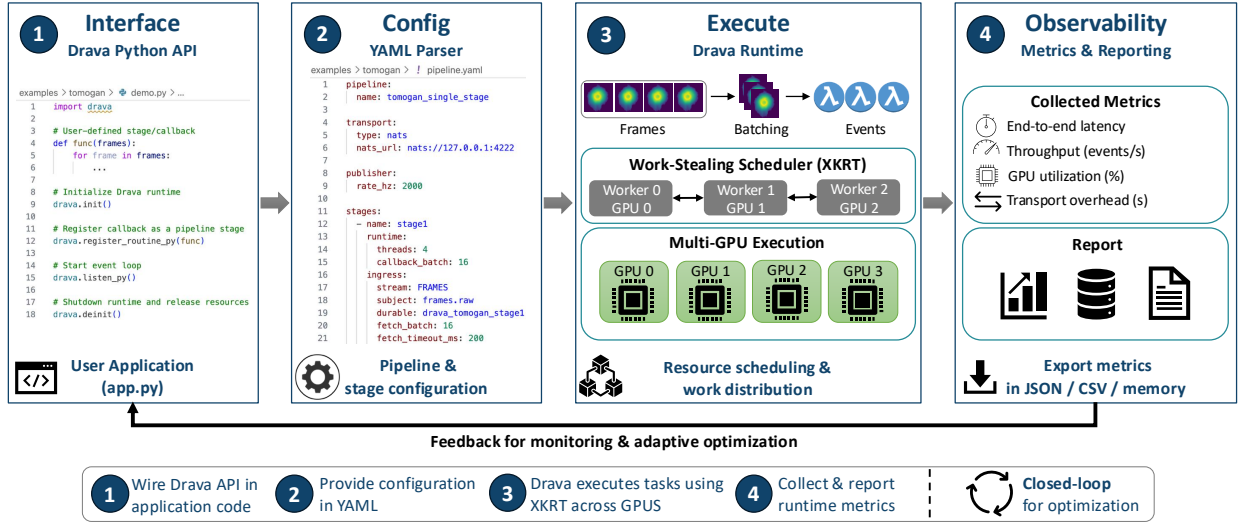


Fig. 4: End-to-end application workflow. Users define stage callbacks through the Python or C API, provide stage and transport configuration in YAML, execute the pipeline through the DRAVA runtime, and collect per-stage metrics that can feed back into the configuration for closed-loop optimization.

the last callback or publish event: $T_i^{\text{stage}} = t_i^{\text{last}} - t_i^{\text{first}}$. It yields a stage throughput of $fps_i = n_i / T_i^{\text{stage}}$, where n_i is the number of non-EOS frames processed.

Stage-latency samples measure the time from callback-batch construction to publish completion, capturing dispatch-to-output delay rather than transport or queuing time. Metrics are exposed programmatically to users, and can also be printed as structured log lines at the end of the stream.

D. Designing and Optimizing DRAVA Applications

Figure 4 summarizes the end-to-end path from application definition to runtime measurement in four stages:

- 1) *Interface*: The application registers stages and callbacks via the C or Python API. It defines only the scientific transformation at each stage.
- 2) *Configuration*: A YAML file binds each stage to its transport routes and runtime parameters. The runtime reads this file at initialization, so changing execution policy requires no code modification.
- 3) *Execution*: The runtime drives the pipeline’s execution: the I/O thread fetches events, the microbatching layer accumulates and flushes batches, and XKRT dispatches callback tasks across the worker pool. Stages progress as input data become available: not on a global clock.
- 4) *Observability*: Per-stage metrics (Section II-C3) are collected throughout execution and emitted on EoS events.

Following this workflow, we implemented two motivating applications on top of DRAVA: PtychoNN and TomoGAN. Figure 5 shows their pipeline structures. PtychoNN uses a two-stage inference-and-stitching pipeline, whereas TomoGAN uses a single inference stage for slice denoising.

PtychoNN [21] performs ptychographic phase retrieval through a convolutional neural network with two output heads (amplitude and phase). Its DRAVA pipeline has two

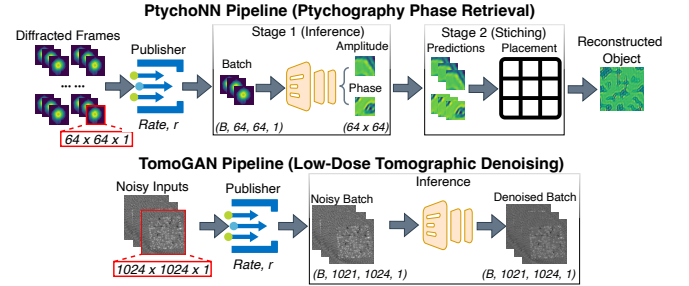


Fig. 5: Structures of PtychoNN and TomoGAN.

stages. Stage 1 receives $64 \times 64 \times 1$ float32 diffraction patches (16 384 bytes each), batches them into a $(B, 64, 64, 1)$ tensor, and runs prediction on publicly available pretrained Keras/HDF5 checkpoint [22]. Predictions are serialized into a binary wire format and published to Stage 2 in configurable chunks. Stage 2 accumulates predictions and performs overlap-add stitching over a 60×60 scan grid, cropping each patch to a 12×12 center region ($4 \times$ the 3-pixel scan step).

TomoGAN [23], [24] performs low-dose tomographic image denoising using the generator of a trained GAN. Its DRAVA pipeline is single-stage: $1024 \times 1024 \times 1$ float32 slices are streamed to a consumer that batches them into a $(B, 1024, 1024, 1)$ tensor and runs AI inference on a pretrained model. Denoised outputs are written to HDF5 with optional MSE evaluation against ground-truth slices.

E. Summary

DRAVA is a multi-layered runtime system for the parallel processing of high-throughput event-driven messages. It includes multiple configuration parameters. We previously introduced generic parameters such as the number of consumer threads (N) or microbatching parameters (B^{cb} , τ)—

and all metrics in Table I. Applications may also define their own parameters, as illustrated with the inference batching B . Additionally, DRAVA also exposes observability through performance (frame rates) and energy efficiency metrics.

This structure creates a direct path from runtime measurement to configuration adaptation: an external optimizer can consume the emitted metrics and propose new configurations for subsequent runs, enabling systematic exploration of the batching, concurrency, and transport parameter space. We propose to explore this space in the following section.

III. AGENTIC HYPERPARAMETER OPTIMIZATION

DRAVA pipelines expose a configuration space in which stage-level parameters interact in non-obvious ways. Increasing the number of threads for one stage may shift the bottleneck to a downstream stage, while larger callback batches improve per-callback amortization at the cost of additional queuing delay. Exhaustively evaluating combinations is time-consuming and expensive because each configuration requires a complete execution of the multi-stage pipeline. We therefore use an agentic autotuning workflow, shown in Figure 6, that performs guided exploration of the configuration space.

The recent HPC autotuning tool *ytopt* has shown that Bayesian optimization with surrogate models can efficiently explore millions of configurations while reducing runtime, energy, and energy-delay product at large scale [25], [26]. Our agent uses *ytopt* to select configurations from the parameter space, generates a YAML configuration, executes the DRAVA pipeline, records measured metrics, and updates the *ytopt* optimization loop until a predefined evaluation budget or wall-clock budget is exhausted, in order to identify the best configuration under a user-defined objective. The resulting configurations and measured metrics are stored as a performance database for subsequent analysis.

A. Configuration and Metric Space

The agent operates on the configuration parameter space

$$\theta = (B, N_1, \dots, N_n, B_1^{cb}, \dots, B_n^{cb}, r), \quad (1)$$

where B is the application-specific inference batch size, N_i is the thread count assigned to stage i , B_i^{cb} is the callback batch threshold for stage i , and r is the publisher rate in Hz. The special case $r = 0$ denotes unconstrained publishing rate. For each sampled configuration, the benchmark driver generates a concrete YAML configuration file, launches the configured DRAVA pipeline, and records the structured metric vector

$$\mathbf{m}(\theta) = (T_{e2e}, T_1, \dots, T_n, fps_1, \dots, fps_n, E), \quad (2)$$

where T_{e2e} is the end-to-end pipeline latency, T_i is the latency of stage i , fps_i is the throughput of stage i , and E denotes an optional energy or power-derived metric when available.

B. *ytopt*-Guided Exploration

The agent performs closed-loop Bayesian optimization using *ytopt*. At each iteration, *ytopt* selects a candidate configuration θ_i from the user-defined parameter space in Equation 1.

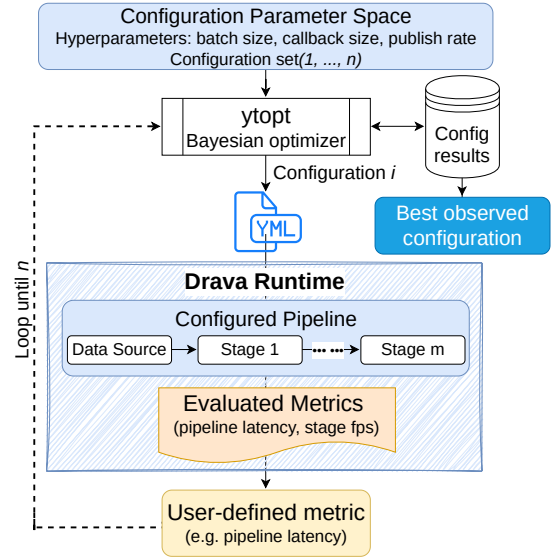


Fig. 6: Agentic autotuning workflow for DRAVA using *ytopt* to explore user-defined parameter space and report the best observed configuration.

The benchmark runner materializes this configuration as a YAML file, executes the corresponding application pipeline using DRAVA, and collects the measured metric vector in Equation 2 from the benchmark summary. The user specifies a scalar objective over the measured metrics,

$$J(\theta) = g(\mathbf{m}(\theta)),$$

where $g(\cdot)$ selects or combines one or more metrics from the metric vector. For example, the objective may minimize end-to-end latency, minimize a specific stage latency, maximize a stage-level throughput, or optimize a weighted latency-throughput tradeoff. When the selected objective is naturally expressed as a maximization, the benchmark driver converts it to the minimization form expected by the optimizer; for example, maximizing stage- k throughput is represented as $J(\theta) = -fps_k(\theta)$.

Because each objective evaluation requires a complete pipeline run, the agent uses Bayesian optimization to reduce the number of measured configurations. The optimizer builds an observation-driven model of the objective and uses it to select the next configuration by balancing exploitation of promising regions with exploration of uncertain regions. As shown in Figure 6, each *ytopt* iteration consists of four implementation steps: candidate selection, YAML materialization, measured pipeline execution, and objective/metric logging into the performance database. After each execution, the evaluated objective $(\theta_i, J(\theta_i))$ and the full measured metric vector $\mathbf{m}(\theta_i)$ are appended to the database and returned to *ytopt* for the next iteration. *ytopt* uses the accumulated observations to select the next candidate configuration, balancing exploitation of previously good regions with exploration of unevaluated regions of the parameter space. This process continues until the predefined evaluation budget, `max-evals`, is exhausted.

The agent returns the best observed configuration under the user-defined measured objective,

$$\theta^* = \arg \min_{i \leq \text{max-evals}} J(\theta_i),$$

where the minimization is taken over all configurations that were actually executed during the ytopt search. Thus, θ^* is not a predicted optimum over the full parameter space, but the best measured configuration found within the available evaluation budget. The agent also stores the complete configuration–metric pair as a record for subsequent analysis.

IV. EVALUATION

We evaluate DRAVA on streaming X-ray science workloads that combine GPU inference, CPU postprocessing, and inter-stage data movement. We begin with a synthetic dataflow benchmark that isolates the runtime message path and establishes a message-rate ceiling (Section IV-B). We then add real PtychoNN model inference to this path and compare DRAVA against a direct PvaPy baseline on the same input stream (Section IV-C). We then study manually selected PtychoNN configurations to characterize the throughput–latency trade-off in a two-stage pipeline (Section IV-D). We next use DRAVA’s observability to attribute this behavior to specific runtime mechanisms (Section IV-E). We evaluate an agentic configuration-search loop that uses Bayesian optimization to find high-performing DRAVA configurations for PtychoNN under a bounded evaluation budget (Section IV-F).

A. Experiment Environment

TABLE II: Experiment platform and software stack.

Category	Configuration
System	JLSE, Argonne National Laboratory [27]
CPU	Dual-socket AMD EPYC 7532, 32 cores/64 threads
Memory	256 GB DDR4
GPU	NVIDIA A100 40 GB PCIe
Compiler	LLVM/Clang 23.0.0git, C++20
GPU stack	CUDA 12.3, cuDNN 9.19
Python stack	Free-threaded CPython 3.13, NumPy 2.3
ML stack	TensorFlow 2.20, Keras 3.12
Runtime deps.	XKRT, yaml-cpp, NATS C, SWIG 4.4.1
Transport (NATS)	NATS Server v2.11 with JetStream

Table II summarizes the platform and software stack used for all experiments. The experiments run on a single node to isolate runtime behavior from network variability. We evaluate DRAVA on the PtychoNN and TomoGAN streaming inference workloads introduced in Section II-D. For PtychoNN, the publisher uses synthetic replay: 3,600 deterministic PRNG-generated frames matching the model input shape (64×64 , single-channel, float32) are cycled to reach the target frame count. The pretrained PtychoNN weights are taken from the public repository [22]. For TomoGAN, the publisher replays 16 preprocessed noisy/ground-truth slice pairs at 1024×1024 resolution. Both workloads use pretrained models only for inference; no training or fine-tuning occurs. Each stage performs two warmup inference passes on zero-filled tensors

before measurement to initialize GPU kernels and backend autotuning.

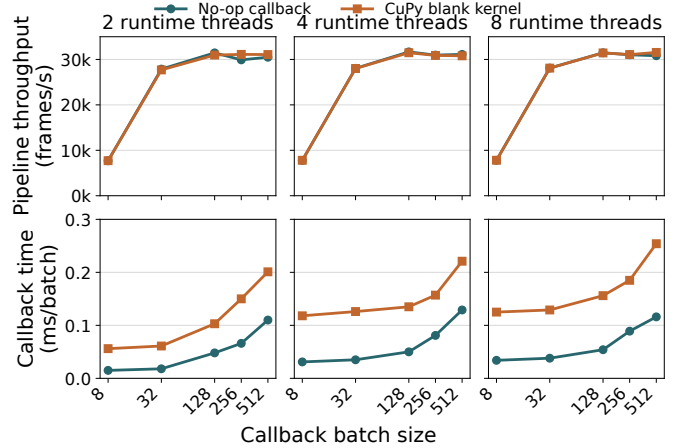


Fig. 7: Runtime message-rate ceiling for a cached one-byte dataflow. Throughput saturates near 31K frames/s, and a minimal CUDA launch increases callback time without materially changing end-to-end throughput.

B. Runtime Message-Rate Ceiling

To separate runtime effects from application-specific computation, we evaluate a synthetic dataflow benchmark that preserves the normal publisher, JetStream transport, DRAVA callback dispatch, callback batching, and end-of-stream handling, but removes dataset loading, tensor construction, model inference, and output materialization. The publisher repeatedly sends a cached one-byte payload for 100,000 frames. The consumer validates each payload and executes one of two callback bodies: a no-op callback, or a callback that launches and synchronizes one empty CuPy CUDA kernel per callback batch. The second mode does not model useful GPU computation; it measures the incremental effect of a minimum GPU launch on the same runtime path where a DRAVA task waits synchronously for GPU kernel launches.

The second mode does not model useful GPU computation; it measures the incremental effect of launching a minimum GPU task on the same runtime path, including the synchronous wait that a DRAVA task performs for the GPU kernel to complete.

Figure 7 shows two effects. First, throughput is primarily controlled by callback batch size. With a no-op callback, throughput increases from about $7.7\text{--}7.8 \times 10^3$ frames/s at batch size 8 to about 2.8×10^4 frames/s at batch size 32, then saturates around 3.1×10^4 frames/s for batch sizes 128–512. Increasing the runtime thread count from 2 to 8 provides little additional throughput, indicating that this synthetic configuration is limited by the transport and dispatch path rather than by callback compute.

Second, adding the empty CUDA launch increases the measured callback time but does not materially change end-to-end throughput. For example, at batch size 512 the av-

erage callback time rises from 0.110–0.129 ms in the no-op case to 0.201–0.254 ms with the CuPy launch, while pipeline throughput remains near 3.1×10^4 frames/s. This result establishes an upper bound for the message-rate regime used by the application experiments: once callback batches are large enough, the runtime path sustains roughly 31K frames/s even when each batch includes a minimum GPU launch. Real GPU computation is evaluated separately in the PtychoNN and TomoGAN application studies.

C. Baseline Comparison with PvaPy

We next add PtychoNN model inference to the runtime message path using a single-stage workload. The baseline uses PvaPy directly: a PvaPy server updates one PVA record with raw PtychoNN frames, and a monitor callback batches frames, invokes the same TensorFlow/Keras model, and publishes Stage 1 predictions. DRAVA uses the same synthetic frame pool, pretrained model, inference batch sweep $B \in \{128, 256, 512\}$, but replaces the direct record monitor with DRAVA’s JetStream-backed ingress and runtime callback path.

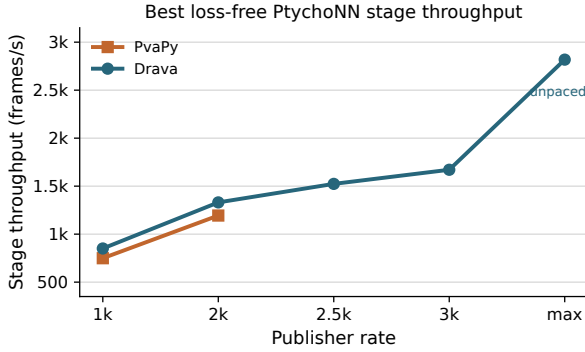


Fig. 8: Throughput evaluation for a single-stage PtychoNN inference stream. Lossy PvaPy runs above 2 kHz are excluded.

We evaluate both paced and unpaced publisher modes. In paced runs, the publisher emits frames at fixed rates from 1 to 3 kHz. In the unpaced run, the publisher removes this rate limiter and sends the frames as fast as the ingress path accepts it, stressing the runtime’s burst-ingest capacity rather than modeling a fixed detector frame rate.

Figure 8 reports the best loss-free stage throughput over the batch sweep. In paced runs, PvaPy remains loss-free up to 2 kHz, while DRAVA is 1.11–1.14 $\times$ faster at the same rates. Above 2 kHz, PvaPy drops frames, missing 400 frames at 2.5 kHz and 784 at 3 kHz, so we exclude these timeout-dominated lossy runs from the throughput curve. DRAVA remains loss-free at all measured rates and reaches 1.28–1.40 $\times$ the best loss-free PvaPy throughput at 2.5–3 kHz. In unpaced mode, DRAVA processes the same 3,600-frame stream without loss, draining the stage at 2818 frames/s, 2.36 $\times$ the best loss-free PvaPy throughput.

D. Throughput–Latency Trade-off in Manual Configurations

Manual tuning is a common starting point for deploying a streaming pipeline, particularly when the most visible cost is

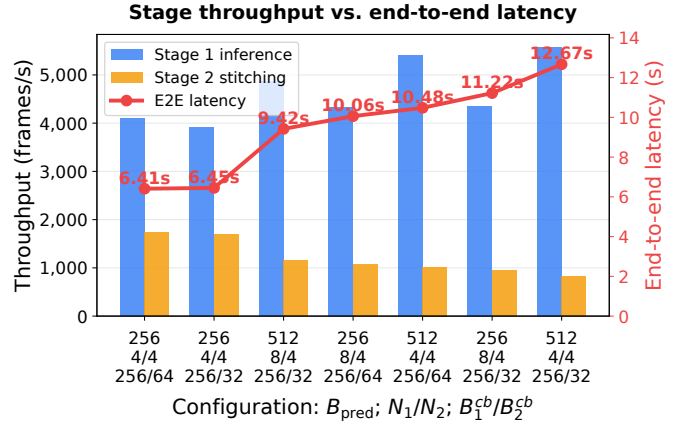


Fig. 9: Stage throughput and end-to-end latency for manually selected PtychoNN configurations.

GPU inference. We characterize a small set of manually selected PtychoNN two-stage configurations and report Stage 1 inference throughput, Stage 2 stitching throughput, and end-to-end latency across seven configurations that vary inference batch size, per-stage thread counts, and callback batching. Figure 9 shows that the fastest individual stage is not sufficient to predict the best pipeline configuration. The configuration with the highest Stage 1 throughput, 5560.56 frames/s, has the worst end-to-end latency, 12.67 s, because Stage 2 falls to 828.03 frames/s. In contrast, the two best end-to-end configurations both use batch size 256 with four threads per stage and keep Stage 2 above 1700 frames/s, reaching 6.41–6.45 s despite lower Stage 1 throughput.

These measurements indicate that the pipeline is governed by cross-stage balance rather than by maximizing GPU inference throughput alone. A manual heuristic that targets the GPU stage in isolation selects a configuration that is roughly 2 $\times$ slower end-to-end than the best observed point. The result provides a small but representative example of the broader configuration problem: callback batching, thread allocation, inference batch size, and stage-boundary movement interact across the full pipeline.

E. Observability-Guided Runtime Tuning

To understand the mechanism behind the manual tuning results, we use the same two-stage PtychoNN pipeline with DRAVA’s runtime instrumentation enabled. Stage 1 performs TensorFlow/Keras GPU inference, and Stage 2 performs CPU stitching, so the pipeline combines an ML stage with a non-ML postprocessing stage. Both stages use four worker threads, the fetch timeout is fixed at 200 ms, publisher rate limiting is disabled, and each configuration processes 10,000 frames while sweeping $B \in \{32, 64, 128, 256, 512\}$. In this experiment, B is both the Stage 1 inference batch size and the callback-batch threshold for both stages. At $B=128$, DRAVA accumulates 128 raw frame payloads from the Stage 1 input stream, dispatches them as a callback task, and the callback forms a (128, 64, 64, 1) tensor before invoking TensorFlow

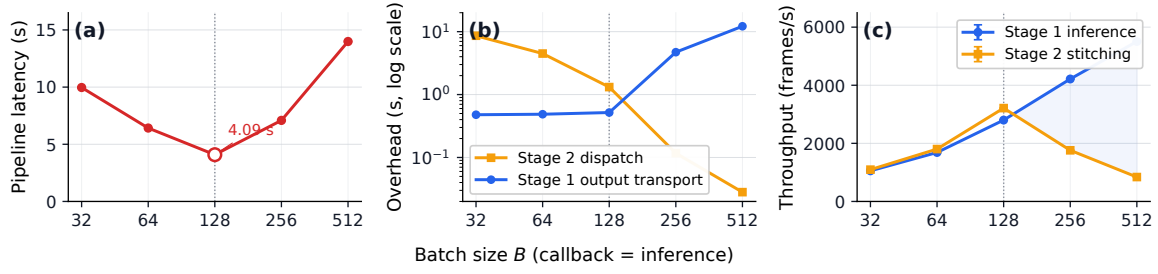


Fig. 10: Runtime observability for batch-size tuning in the mixed GPU/CPU PtychoNN pipeline. (a) End-to-end latency is minimized at $B=128$. (b) Runtime components identify the shift from dispatch overhead to stage-boundary output transport as B increases. (c) Stage throughput shows why the full-pipeline optimum differs from the GPU inference optimum.

inference on the GPU. The Stage 1 callback then publishes prediction payloads downstream in 16-frame chunks, which Stage 2 consumes concurrently; the final stitching computation runs after EOS once the complete scan has been accumulated. Figure 10(a) shows that this streaming pipeline is minimized at $B=128$: end-to-end latency drops from 9.97 s at $B=32$ to 4.09 s at $B=128$, but rises to 13.99 s at $B=512$.

The instrumentation separates this curve into runtime components. Figure 10(b) shows that the dominant component changes with B . At small batches, Stage 2 dispatch dominates and falls from 8.61 s at $B=32$ to 1.31 s at $B=128$, indicating that fixed per-batch callback submission overhead is amortized. At large batches, the dominant cost shifts to Stage 1 output transport, which stays near 0.5 s through $B=128$ but increases to 4.72 s at $B=256$ and 12.17 s at $B=512$. This is consistent with the Stage 1 execution path: larger inference batches reduce callback frequency, but each callback emits larger prediction data across the stage boundary. The remaining runtime mechanisms are not the limiting factor in this sweep: microbatch wait is effectively zero, Stage 1 publish remains below 0.1 s, and Stage 2 does not republish. The measured overhead is therefore localized to two system-level mechanisms, callback dispatch and stage-boundary data movement, rather than appearing as an undifferentiated framework tax.

Figure 10(c) relates the component measurements to pipeline throughput. Stage 1 throughput increases monotonically from 1054.05 to 5505.87 frames/s as B grows, so the GPU inference stage alone favors the largest batch. The full pipeline follows a different optimum: Stage 2 peaks at 3213.35 frames/s at $B=128$ and then drops to 839.99 frames/s at $B=512$, while end-to-end throughput peaks at 2446.29 frames/s at $B=128$, a $2.44\times$ improvement over $B=32$. DRAVA’s measurements expose how callback batching, GPU inference, stage-boundary transport, and downstream CPU stitching jointly determine the operating point.

F. Agentic Configuration Search

The preceding PtychoNN experiments sample only a small part of a larger configuration surface. To evaluate whether the full joint space can be searched under a practical budget, we use an agentic configuration-search workflow for the two-

stage PtychoNN pipeline. The agent uses ytopt as its decision policy and the DRAVA benchmark harness as its execution environment. At each iteration, the agent proposes a categorical pipeline configuration, executes the benchmark, observes the measured objective, records the result, and updates the optimizer state before selecting the next configuration.

The agent is instantiated with the following task prompt:

Prompt: Minimize the end-to-end latency of the two-stage PtychoNN DRAVA pipeline. Search the allowed categorical configuration space. For each proposed configuration, generate the pipeline configuration, execute the benchmark harness, parse the resulting metrics, and update the optimizer with the measured latency. Continue until the evaluation budget is exhausted. Return the best observed configuration and the ranked performance database.

The search space contains seven categorical knobs. The first is the stage 1 model-inference batch size $B_{\text{pred}} \in \{128, 256, 512, 1024\}$, which controls the TensorFlow model.predict batch size after frames have been delivered to the stage 1 callback. The remaining knobs are the stage 1 and stage 2 thread counts $N_1 \in \{4, 8, 12, 16\}$ and $N_2 \in \{2, 4, 8, 12\}$, the stage 1 and stage 2 callback batch sizes $B_1^{cb} \in \{128, 256, 512, 1024\}$ and $B_2^{cb} \in \{32, 64, 128, 256\}$, the publisher rate cap $r \in \{0, 1000, 2000, 4000\}$ Hz, and the fetch timeout $\tau \in \{100, 200, 500\}$ ms. Here $r = 0$ denotes unconstrained publishing rather than a zero-rate publisher. The full Cartesian product contains 12,288 configurations. We give the agent a budget of 64 evaluations, or 0.52% of the full grid. The first 12 evaluations initialize ytopt, and the remaining evaluations are selected using ytopt’s surrogate-guided Bayesian optimization loop. The objective is to minimize measured end-to-end latency, T_{e2e} .

Figure 11 summarizes the closed-loop search process. The agent quickly identifies a low-latency region characterized by $B_{\text{pred}} = 256$, low stage 1 thread count, unconstrained publishing, and small stage 2 callback batches. Table III reports the best configurations found during the 64-evaluation run.

The best observed configuration uses $B_{\text{pred}} = 256$, $N_1 = 4$, $N_2 = 2$, $B_1^{cb} = 512$, $B_2^{cb} = 32$, unconstrained publishing, and $\tau = 200$ ms, yielding $T_{e2e} = 2.86$ s. This is $2.24\times$ faster than the best manually selected configuration in Section IV-D and is reached after evaluating only 0.52% of the joint configuration

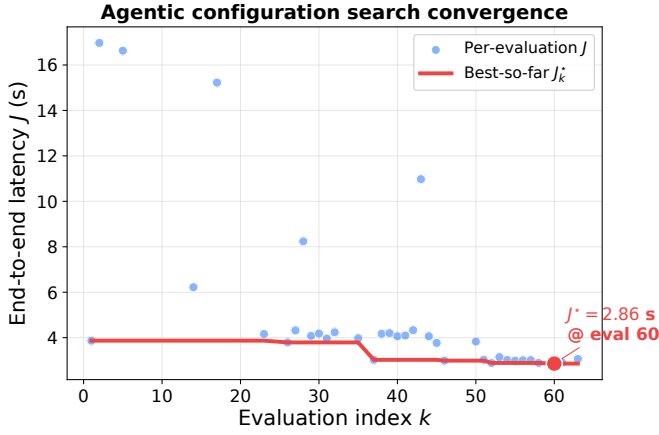


Fig. 11: Convergence of the ytopt-based configuration search. Points show evaluated configurations; the line shows the best latency observed so far.

TABLE III: Top configurations found by the configuration-search agent. Here B_{pred} is the inference batch size, N_1/N_2 are stage thread counts, B_1^{cb}/B_2^{cb} are callback batch sizes, r is publisher rate (0 means uncapped), and τ is fetch timeout.

Rank	B_{pred}	N_1/N_2	B_1^{cb}/B_2^{cb}	r	τ	T_{e2e} (s)
1	256	4/2	512/32	0	200	2.86
2	256	4/2	1024/32	0	100	2.89
3	256	4/4	512/32	0	100	2.89
4	256	4/8	512/32	0	200	2.91
5	256	4/8	1024/32	0	200	2.99

space. The top configurations all choose the same model-inference batch, the lowest stage 1 thread count, the smallest stage 2 callback batch, and unconstrained publishing, while varying stage 2 thread count, stage 1 callback batch size, and timeout. This concentration indicates that the search budget is not merely sampling uniformly; it is converging on a low-latency region shaped primarily by stage 1 model-inference batching, stage 2 callback batching, and the ingress rate cap.

V. ENERGY EFFICIENCY OF STREAMING INFERENCE

We next use DRAVA’s observability to analyze energy efficiency on a second streaming inference workload. This study uses TomoGAN, a generative model that denoises 1024×1024 tomography frames and is representative of streamed image-to-image inference in scientific imaging facilities. Frames arrive through JetStream and are denoised by a TensorFlow TomoGAN model inside the DRAVA stage callback. Figure 12 confirms that this streamed path preserves the inference quality of the original TomoGAN: the DRAVA output is visually close to the ground truth, and the absolute-error map localizes only minor residual differences.

DRAVA’s callback batch and the TensorFlow inference batch are coupled in this stage: increasing the batch size changes both the number of frames delivered per callback and the number of frames submitted to the inference task. The experiment therefore measures the energy effect of the callback–

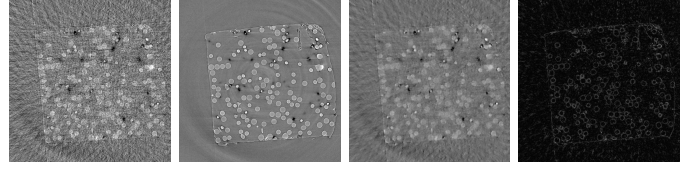


Fig. 12: Qualitative TomoGAN inference result from the DRAVA JetStream pipeline. From left to right: noisy input, ground truth, DRAVA/TomoGAN denoised output using the iteration-500 checkpoint, absolute error between the ground truth and denoised output.

inference batch size, not of transport batching in isolation. We focus on GPU energy because TomoGAN’s denoising work is dominated by TensorFlow inference on the A100, while the CPU side mainly orchestrates streaming, callbacks, and data movement. We stream 512 frames per run over the batch range $\{2, 4, 8, 16\}$, fix the number of XKRT callback threads at two so that batch size is the only varying control, and report the mean of three runs per configuration. During each run, we sample GPU power draw, utilization, and memory use with `nvidia-smi` every 200 ms from publisher start until the DRAVA stage reports its final metrics. The benchmark estimates total GPU energy by integrating the sampled power trace and reports joules per processed frame as

$$E_{\text{frame}} = \frac{E_{\text{GPU}}}{N_{\text{frames}}}, \quad E_{\text{GPU}} \approx \sum_{i=1}^{M-1} \frac{P_i + P_{i+1}}{2} (t_{i+1} - t_i),$$

where P_i is the sampled GPU power at time t_i , M is the number of samples in the measurement window, and N_{frames} is the number of processed frames. The benchmark verifies that the number of published frames matches the number of frames consumed by the stage before computing this value.

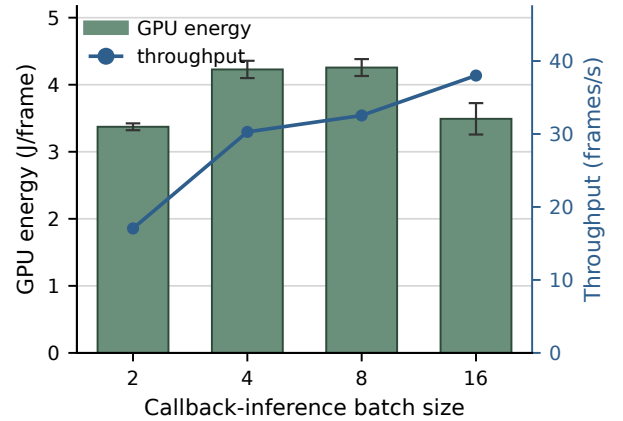


Fig. 13: GPU energy efficiency and throughput for TomoGAN inference. Bars show joules per processed frame (lower is better); the line shows stage throughput (higher is better).

Figure 13 shows that throughput and energy efficiency do not scale identically with batch size. Stage throughput grows monotonically from 17.1 fps at batch 2 to 30.3, 32.5, and

38.0 fps at batches 4, 8, and 16, with the largest gain ($1.77\times$) coming from batch 2 to batch 4; the stage becomes increasingly GPU-bound thereafter. DRAVA overhead, measured as the fraction of wall time spent outside the user inference call, stays low across the sweep (1.8–5.0%), so the runtime is not the limiting factor at any operating point. GPU joules per frame, in contrast, is non-monotonic. Batch 2 is the most energy-efficient point we measured at 3.37 J/frame despite delivering the lowest throughput; batches 4 and 8 both consume roughly 4.2 J/frame (a 25–26% energy premium) while improving throughput by 1.8–1.9 $\times$; and batch 16 recovers to 3.49 J/frame as the A100 becomes sufficiently occupied to amortize the additional GPU work, memory traffic, and kernel execution cost.

These results complement the PtychoNN latency and throughput evaluation by showing that the same runtime-facing control, batch size, can also change the energy profile of a streamed scientific inference workload. Batch 16 provides the highest throughput, while batch 2 provides the lowest measured joules per frame; batches 4 and 8 trade additional energy for intermediate throughput. Reporting joules per frame alongside throughput makes these operating points explicit for facilities that must balance turnaround time with accelerator energy use.

VI. RELATED WORK

DRAVA is at the intersection of high-level data pipeline orchestration and low-level event-driven execution. While existing research has explored serverless paradigms for scientific workflows and specialized runtimes for HPC services, there remains a gap for a reusable, open-source component that bridges these layers for streaming data. We review state of the art across two primary dimensions: high-level Function-as-a-Service (FaaS) frameworks, low-level parallel runtime systems for event-driven execution.

A. Data Pipelines and Function-As-A-Service (FaaS)

PvaPy is a Python interface to the EPICS pvAccess protocol and has been used as the basis for a real-time streaming framework for X-ray detector data [10], [28]. The framework moves detector streams directly into Python applications, supports local or remote processing, and returns results to users during an experiment. funcX (Globus Compute¹) [29] is a distributed function as a service (FaaS) platform for scalable remote function execution. It notably supports the execution of remote Python functions via serialization of objects, batching, and addresses security aspects via user authentication. Globus Automate [12] is built upon funcX for event-driven execution of Python functions. These components are tightly intricated within the larger Globus ecosystem (e.g., Globus Auth, Globus Connect, etc.) that emphasizes flexibility for the management of scientific data, from sensors to edge computing or super-computer systems.

Numaflow [30] is a Kubernetes-based serverless platform for running scalable event-driven applications. Alike DRAVA,

it supports multiple data transport backends such as Kafka [31] or Jetstream [20], based on publisher/subscriber models to scale onto distributed systems. However, they operate on different granularities and latencies: Numaflow targets coarse-grain (milliseconds-seconds) functions with virtualization/containerization on cloud systems using Kubernetes, while DRAVA enables fine-grain (nanoseconds-microseconds) functions by natively running on modern AI/HPC multi-device systems using XKRT [13]. Both support multiple programming languages (Python, Julia, C, C++).

B. Runtime Systems for Event-Driven Systems

Mochi [32] is a collection of software components to develop HPC data services. Its `mochi-abt-io` relies on the user-level threading library Argobots [33] to implement lightweight POSIX I/O APIs. Its `mochi-mercury` component provides efficient (high-throughput, low-latency) communications components on supercomputers’ highly-specialized networks (e.g., RDMA, zero-copies, etc.). Mofka [34] implements an event-driven publisher/subscriber model that scales to supercomputers using these Mochi components. DRAVA supports two backends for multi-process communications: (1) `AF_UNIX` sockets for efficient intra-node IPCs and (2) Jetstream (NATS) [20] for inter-node communications that implements a publisher/subscriber model, alike Mofka — adding a Mofka transport backend is left as future work to enhance performance on supercomputers.

XKRT [13] parallel runtime system to efficiently schedule work on multi-device systems — such as multi-GPU compute nodes found in AI and HPC clusters. XKRT provides low-level building blocks for compute node-level parallelism, including tasking, NUMA-aware work-stealing, threading, device management, and automatic multi-device memory management (allocation, copies, eviction, etc.) — on which DRAVA relies to implement its event-driven programming model.

VII. CONCLUSION

This paper introduces DRAVA: an event-driven runtime for scientific streaming workflows that sustains unpaced input data rates targeting next-generation detectors. We presented its (i) event-driven programming model, (ii) implementation as a runtime system, (iii) agentic workflow for optimizing deployment—via guided exploration using `yopt`—and (iv) integration and evaluation on representative ptychography and tomography scientific pipelines. Results show $2.36\times$ higher frame throughput against state-of-the-art.

Future work will extend DRAVA to distributed multi-node streaming and energy-aware online adaptation, including dynamic concurrency throttling of thread counts and other application/runtime knobs. We also plan to evaluate DRAVA on emerging data-centric architectures, including ASIC-based accelerators and the cycle-latency memory accelerator Up-Down [18], to study how event-driven runtime abstractions map to hardware designed for irregular, communication-intensive workflows and PB/s-scale data rates.

¹<https://github.com/globus/globus-compute>

REFERENCES

- [1] A. V. Babu, T. Zhou, S. Kandel, T. Bicer, Z. Liu, W. Judge, D. J. Ching, Y. Jiang, S. Veseli, S. Henke *et al.*, “Deep learning at the edge enables real-time streaming ptychographic imaging,” *Nature Communications*, vol. 14, no. 1, p. 7059, 2023.
- [2] Argonne National Laboratory, “Advanced photon source strategic plan 2025–2029,” <https://www.aps.anl.gov/The-Advanced-Photon-Source-Strategic-Plan>, 2025, accessed: 2026-05-01.
- [3] H. R. Stueber, A. Y. Pan, T. Chattopadhyay, S. W. Allen, M. W. Bautz, K. Donlon, C. E. Grant, S. Herrmann, B. LaMarr, A. Malonis *et al.*, “Fast, low noise ccd systems for future strategic x-ray missions,” in *UV, X-Ray, and Gamma-Ray Space Instrumentation for Astronomy XXIV*, vol. 13625. SPIE, 2025, pp. 501–516.
- [4] J.-W. Burlage, F. Marone, D. M. Pelt, W. J. Palenstijn, M. Stampanoni, K. J. Batenburg, and C. M. Schlepütz, “Real-time reconstruction and visualisation towards dynamic feedback control during time-resolved tomography experiments at tomcat,” *Scientific reports*, vol. 9, no. 1, p. 18379, 2019.
- [5] D. Windisch, O. Knodel, G. Juckeland, U. Hampel, and A. Bieberle, “Fpga-based real-time data acquisition for ultrafast x-ray computed tomography,” *IEEE Transactions on Nuclear Science*, vol. 68, no. 12, pp. 2779–2786, 2021.
- [6] E. Vernon, G. De Geronimo, J. Baldwin, W. Chen, J. Fried, G. Giacomini, A. Kuczewski, J. Kuczewski, J. Mead, A. Miceli *et al.*, “Development of a high-rate front-end ASIC for x-ray spectroscopy and diffraction applications,” *IEEE Transactions on Nuclear Science*, vol. 67, no. 4, pp. 752–759, 2020.
- [7] M. Minuti, L. Baldini, R. Bellazzini, A. Brez, M. Ceccanti, F. Krummenacher, L. Latronico, L. Lucchesi, A. Manfreda, L. Orsini *et al.*, “Xpol-iii: A new-generation vlsi CMOS ASIC for high-throughput x-ray polarimetry,” *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, vol. 1046, p. 167674, 2023.
- [8] A. Mozzanica, M. Andrä, R. Barten, A. Bergamaschi, S. Chirioti, M. Brückner, R. Dinapoli, E. Fröjd, D. Greiffenberg, F. Leonarski *et al.*, “The jungfrau detector for applications at synchrotron light sources and XFELs,” *Synchrotron Radiation News*, vol. 31, no. 6, pp. 16–20, 2018.
- [9] V. Radicci, A. Bergamaschi, R. Dinapoli, D. Greiffenberg, B. Henrich, I. Johnson, A. Mozzanica, B. Schmitt, and X. Shi, “Eiger a new single photon counting detector for x-ray applications: performance of the chip,” *Journal of Instrumentation*, vol. 7, no. 02, pp. C02019–C02019, 2012.
- [10] S. Veseli, J. Hammonds, S. Henke, H. Parraga, B. Frosik, and N. Schwarz, “Pvapy streaming framework for real-time data processing,” *Synchrotron Radiation*, vol. 32, no. 3, 2025.
- [11] Y. Kim, S. Park, S. Perarnau, and A. Raj, “Charon: An end-to-end infrastructure for connecting ai@edge to hpc,” in *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2025, pp. 1112–1119.
- [12] R. Vescovi, R. Chard, N. D. Saint, B. Blaiszik, J. Pruyne, T. Bicer, A. Lavens, Z. Liu, M. E. Papka, S. Narayanan, N. Schwarz, K. Chard, and I. T. Foster, “Linking scientific instruments and computation: Patterns, technologies, and experiences,” *Patterns*, vol. 3, no. 10, 2022. [Online]. Available: <https://doi.org/10.1016/j.patter.2022.100606>
- [13] R. Pereira, S. Perarnau, P.-E. Polet, B. Videau, T. Applencourt, and T. Gautier, “XKRT: a Runtime System for Macro-Dataflow Programming on Multi-Devices Architectures,” 2026.
- [14] NATS.io, “Nats concepts: Jetstream,” 2021, accessed: 2026-04-25. [Online]. Available: <https://docs.nats.io/nats-concepts/jetstream>
- [15] U.S. Department of Energy, Office of Science, “Department of Energy Announces \$179 Million for Microelectronics Science Research Centers,” <https://www.energy.gov/science/articles/department-energy-announces-179-million-microelectronics-science-research-centers>, Dec. 2024, accessed: May 21, 2026.
- [16] Microelectronics Energy-Efficiency Research Center for Advanced Technologies, “BIA: A Co-Design Methodology: Focus on Energy Efficiency,” <https://meercat-center.org/projects/bia>, 2026, accessed: 2026-05-21.
- [17] W. Olin-Ammertorp, X. Wu, and A. A. Chien, “Modeling performance of data collection systems for high-energy physics,” *APL Machine Learning*, vol. 2, no. 4, 2024.
- [18] A. A. Chien, C. Colley, J. Ding, A. Fell, D. F. Gleich, H. Hoffmann, M. Jeje, R. Khandelwal, Y. Li, J. M. Monsalve-Diaz, M. Nourian, A. Rajasukumar, J. Su, T. Su, Y. Wang, W. Wang, R. Xu, and T. Zhang, “Updown: A supercomputer co-designed for scalable graph processing,” *IEEE Transactions on Parallel and Distributed Systems*, pp. 1–19, 2026.
- [19] S. Veseli, J. Hammonds, S. Henke, M. Miller, H. Parraga, I. Baldin, D. Howard, Y. Kumar, and N. Schwarz, “Streaming x-ray detector data to remote facilities using ejat,” in *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC Workshops ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 2336–2346. [Online]. Available: <https://doi.org/10.1145/3731599.3767591>
- [20] R. Tudoran, O. Nano, I. Santos, A. Costan, H. Soncu, L. Bougé, and G. Antoniu, “Jetstream: enabling high performance event streaming across cloud data-centers,” in *Proceedings of the 8th ACM International Conference on Distributed Event-Based Systems*, ser. DEBS ’14. New York, NY, USA: Association for Computing Machinery, 2014, p. 23–34. [Online]. Available: <https://doi.org/10.1145/2611286.2611298>
- [21] M. J. Cherukara, T. Zhou, Y. Nashed, P. Enfedaque, A. Hexemer, R. J. Harder, and M. V. Holt, “Ai-enabled high-resolution scanning coherent diffraction imaging,” *Applied Physics Letters*, vol. 117, no. 4, 2020.
- [22] M. J. Cherukara, “PtychoNN_data,” https://huggingface.co/datasets/mc/herukara/PtychoNN_data, 2024, accessed: 2026-04-27.
- [23] Z. Liu, T. Bicer, R. Kettimuthu, and I. Foster, “Deep learning accelerated light source experiments,” in *2019 IEEE/ACM Third Workshop on Deep Learning on Supercomputers (DLS)*, 2019, pp. 20–28.
- [24] Z. Liu, T. Bicer, R. Kettimuthu, D. Gursoy, F. De Carlo, and I. Foster, “Tomogan: low-dose synchrotron x-ray tomography with generative adversarial networks: discussion,” *Journal of the Optical Society of America A*, vol. 37, no. 3, pp. 422–434, 2020.
- [25] X. Wu, P. Paramasivam, and V. Taylor, “Autotuning apache tvn-based scientific applications using bayesian optimization,” in *Proceedings of the SC’23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, 2023, pp. 29–35.
- [26] X. Wu, P. Balaprakash, M. Kruse, J. Koo, B. Videau, P. Hovland, V. Taylor, B. Geltz, S. Jana, and M. Hall, “ytot: Autotuning scientific applications for energy efficiency at large scales,” *Concurrency and Computation: Practice and Experience*, vol. 37, no. 1, p. e8322, 2025.
- [27] Argonne National Laboratory, “Joint Laboratory for System Evaluation (JLSE),” <https://www.jlse.anl.gov>, 2026, accessed: 2026.
- [28] S. Veseli, “PvaPy: Pvaccess for python,” <https://pypi.org/project/pvapy/>, 2025, version 5.6.0, released August 8, 2025. Python library for EPICS PV Access.
- [29] R. Chard, T. J. Skluzacek, Z. Li, Y. Babuji, A. Woodard, B. Blaiszik, S. Tuecke, I. Foster, and K. Chard, “Serverless supercomputing: High performance function as a service for science,” 2019. [Online]. Available: <https://arxiv.org/abs/1908.04907>
- [30] Numaproj Contributors, “Numaflow: Kubernetes-native platform to run massively parallel data/streaming jobs,” <https://github.com/numaproj/numaflow>, 2026, accessed: 2026-04-27.
- [31] R. Shree, T. Choudhury, S. C. Gupta, and P. Kumar, “Kafka: The modern platform for data management and analysis in big data domain,” in *2017 2nd International Conference on Telecommunication and Networks (TEL-NET)*, 2017, pp. 1–5.
- [32] M. Dorier, P. Carns, R. Ross, S. Snyder, R. Latham, A. Gueroudji, G. Amvrosiadis, C. Cranor, and J. Soumagne, “Extending the mochi methodology to enable dynamic hpc data services,” in *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2024, pp. 414–422.
- [33] S. Seo, A. Amer, P. Balaji, C. Bordage, G. Bosilca, A. Brooks, P. Carns, A. Castello, D. Genet, T. Herault, S. Iwasaki, P. Jindal, L. V. Kale, S. Krishnamoorthy, J. Lifflander, H. Lu, E. Meneses, M. Snir, Y. Sun, K. Taura, and P. Beckman, “Argobots: A lightweight low-level threading and tasking framework,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 3, 10 2017.
- [34] M. Dorier, A. Gueroudji, V. Hayot-Sasson, H. D. Nguyen, S. Ockerman, R. Souza, T. Bicer, H. Pan, P. Carns, K. Chard, R. Chard, M. Gonthier, E. Huerta, B. Lenard, B. Nicolae, P. Patel, J. Wozniak, I. Foster, N. S. Rao, and R. B. Ross, “Toward a persistent event-streaming system for high-performance computing applications,” *Frontiers in High Performance Computing*, vol. Volume 3 - 2025, 2025. [Online]. Available: <https://www.frontiersin.org/journals/high-performance-computing/articles/10.3389/fhpcp.2025.1638203>